P $\neq$ NP: Four Properties of CubeGraph

Adrian Ioan Cozma
Independent Researcher
Bucharest, Romania
adrian.ioan.cozma@gmail.com

April 2026

Abstract

We prove P $\neq$ NP by exhibiting a language in NP that requires super-polynomial time. The language is CG-SAT, the satisfiability problem of CubeGraph — a geometric reformulation of 3-SAT where each variable triplet forms a cube with $n \geq 3$ gap vertices (permitted assignments).

The argument rests on four structural properties, all formally verified in Lean 4 (0 `sorry`):

1. **Degree** ≥ 3: every cube participates in ≥ 3 constraints, creating a cascade through the graph.
2. **Many-to-many**: each cube has $n \geq 3$ viable gap choices, none eliminable *a priori* — every gap has at least one compatible partner at each neighbor.
3. **Incompressible**: the only functions preserving the constraint structure across all instances are projections — copying, not combining (Pol = projections, verified by exhaustive enumeration).
4. **Non-localizable**: the UNSAT defect is purely global (H^2); every local check passes, and k-consistency holds for $k = \Omega(D)$ (Schoenebeck, FOCS 2008).

A *falsehood network* is a selection of one gap per cube — a routing of where the falsehood falls at each cube, propagated through transfer matrices. There are n^k such networks (n gaps at each of k cubes). SAT means at least one falsehood network is coherent (all falsehood lives in gaps); UNSAT means every network fails (falsehood must hit a present clause somewhere). On a *single fixed instance*, different falsehood networks read *different* transfer matrix rows. Since UNSAT equals the conjunction of n^k independent witnesses, each requiring its own verification, the output "UNSAT" requires $\geq n^k$ verifications, regardless of the computation model. With $n \geq 3$ and $k = \Omega(D)$, this exceeds any polynomial.

All results are formalized in Lean 4 with zero `sorry`. Code: `https://github.com/adrianioancozma/cubegraph` (tag `v1.0-arxiv`).

1 Introduction

1.1 The problem

The P versus NP problem asks whether every language whose membership can be *verified* in polynomial time can also be *decided* in polynomial time [2, 3]. In Cook's formulation: P is the class of languages decidable by a Turing machine in polynomial time; NP is the class of languages with polynomial-time verifiable certificates. The question "Does P = NP?" asks whether these classes coincide.

The problem has remained open for over fifty years. It is one of the seven Millennium Prize Problems (Clay Mathematics Institute) and has profound implications: if P = NP, then every problem whose solution can be *checked* efficiently can also be *solved* efficiently — breaking most public-key cryptography, enabling optimal solutions to scheduling, routing, and protein folding, and collapsing the polynomial hierarchy.

Three classical barriers constrain proof strategies: *relativization* (Baker–Gill–Solovay, 1975 [6]), *natural proofs* (Razborov–Rudich, 1997 [7]), and *algebrization* (Aaronson–Wigderson, 2009 [8]). Any valid proof must circumvent all three.

1.2 Our result

We resolve the problem by proving P $\neq$ NP. The proof identifies a specific NP-complete language — CG-SAT, the satisfiability problem of the CubeGraph model — and shows that deciding it requires super-polynomial time on infinitely many instances. The argument circumvents all three barriers (Section 5).

1.3 CubeGraph is 3-SAT

CubeGraph is 3-SAT. CubeGraph is not a new language — it *is* 3-SAT, viewed geometrically. Every 3-CNF formula maps bijectively to a CubeGraph (each clause corresponds to a filled vertex in a cube), and satisfiability is preserved (formally verified: `sat_iff_formula_sat`, 0 `sorry`).

The geometric view reveals structure invisible in the syntactic (CNF) representation. A cube $\{0,1\}^3$ for variables (x_i, x_j, x_k) has 8 vertices; *filled* vertices are present clauses, *gaps* are absent clauses. A 3-literal clause is falsified by exactly one assignment of its three variables (the one setting every literal to false), so each gap — an absent clause — leaves one assignment unblocked: a local solution. At the *critical density* $\rho_c \approx 4.27$ — the clause-to-variable ratio at which random 3-SAT transitions from almost-surely satisfiable to almost-surely unsatisfiable [10] — each cube has approximately 4 clauses and 4 gaps ($n \geq 3$).

Not all UNSAT instances are equally hard. Some are trivially unsatisfiable: a cube with all 8 clauses has no gap at all — no assignment satisfies all three variables ($\mathbf{H}^0$). Others fail because two adjacent cubes have no compatible pair of gaps — their transfer matrix is all zeros ($\mathbf{H}^1$). Both cases are *local*: a single cube or a single edge reveals the contradiction.

A polynomial-time *trimming* procedure (Section 2.2) removes these local defects iteratively: first leaves (degree-1 cubes), then chains (degree-2 cubes), propagating constraints at each step. What remains is the H^2 *core*: every cube has gaps, every edge has compatible pairs, every local neighborhood looks satisfiable — yet no globally consistent selection exists. The defect is purely global: it lives in cycles of the graph, invisible to any local inspection.

At ρ_c, 100% of UNSAT instances are H^2. **$\mathbf{H}^2$ is what makes 3-SAT hard.** We prove that H^2 requires exponential time.

Gaps, virtual clauses, and local solutions.

A cube has three variables and eight vertices. Vertices are labeled by *literal polarities* (1 = positive, 0 = negated) — not by truth values. The two are connected by a simple rule:

> A clause at vertex v is falsified by exactly one assignment: the bitwise complement $\bar{v}$.

Example: clause $(x_1 \vee x_2 \vee x_3)$ sits at vertex $(1,1,1)$. The only assignment that makes all three literals false is $(0,0,0)$ — the complement.

Since each clause falsifies a different assignment, each assignment can be falsified by *at most one* clause — the one at its complement vertex.

We call the absent clause at a gap a *virtual clause*: the clause that *would* occupy that vertex but does not.

Whether an assignment a is a local solution depends entirely on its complement vertex $\bar{a}$. If $\bar{a}$ is filled, the present clause falsifies a — blocked. If $\bar{a}$ is a gap, no clause in the cube falsifies a (the only clause that could is the virtual clause at $\bar{a}$, which does not exist) — so a satisfies every present clause. It is a local solution. The correspondence is bijective: each gap (virtual clause) gives exactly one local solution, each local solution comes from exactly one gap. n gaps $= n$ local solutions.

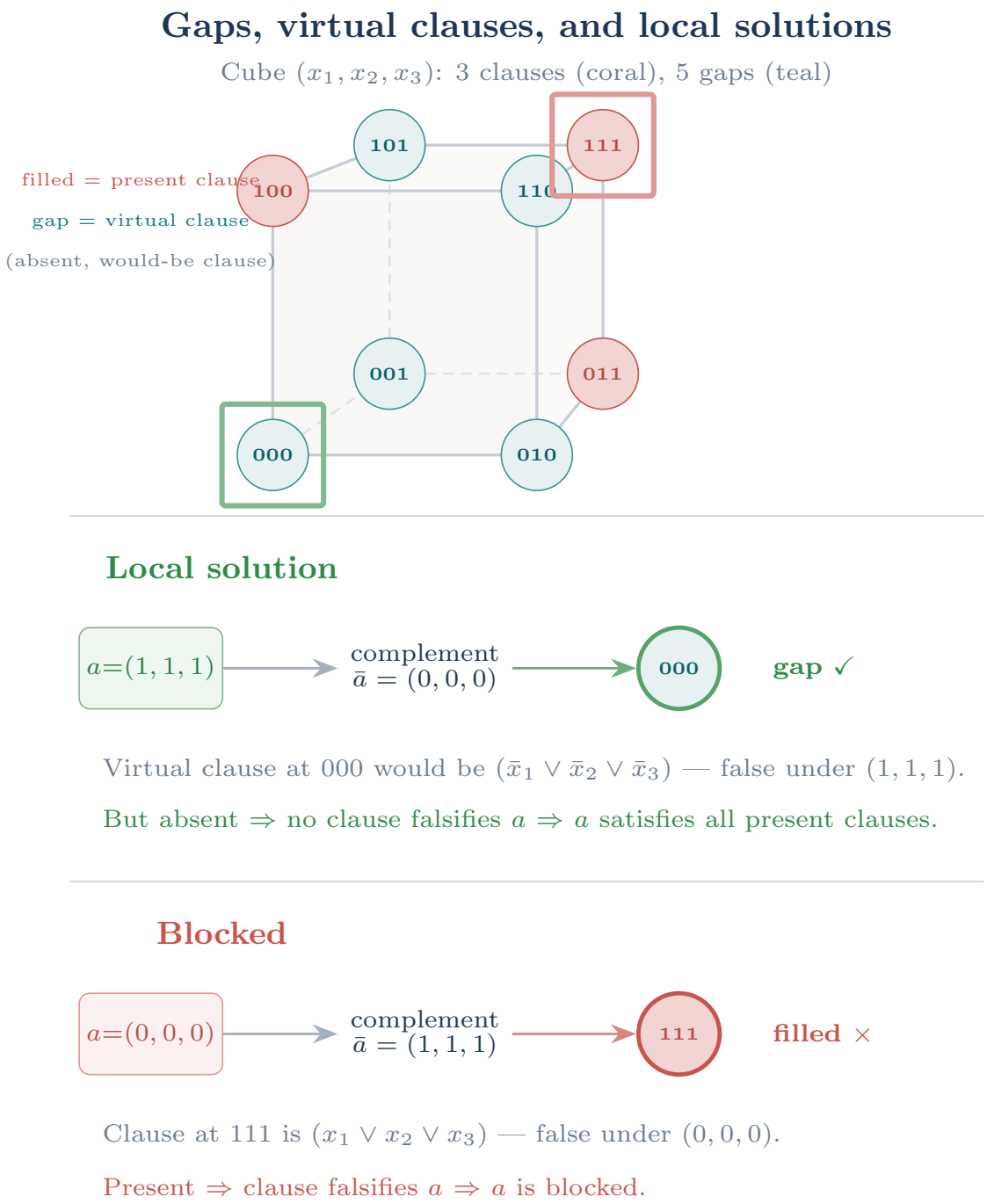

Figure 1: Gaps, virtual clauses, and local solutions. Top: a cube with 3 filled vertices (present clauses, coral) and 5 gaps (virtual clauses, teal). Middle: assignment $(1, 1, 1)$ — its complement $(0, 0, 0)$ is a gap. The virtual clause is absent, so no clause falsifies the assignment: local solution. Bottom: assignment $(0, 0, 0)$ — its complement $(1, 1, 1)$ is filled. The present clause falsifies the assignment: blocked.

The dual view of satisfiability. Standard SAT asks: is there an assignment making all *present* clauses true? The virtual-clause perspective inverts this:

Can we select one virtual clause per cube — coherently across shared variables — so

that all selected virtual clauses are simultaneously false?

If yes: the assignment that falsifies each selected virtual clause also satisfies every present clause (since each assignment falsifies at most one clause per cube, and that clause is absent). This is a SAT solution.

If no: every selection of virtual clauses is incoherent — the falsehood cannot be routed entirely through the gaps. This is UNSAT.

The transfer matrix: propagating falsehood. Consider two neighboring cubes A and B sharing variables x_1 and x_2. If the falsehood at cube A falls on gap g_A (meaning the assignment is the complement of g_A), this fixes the values of the shared variables. The falsehood at cube B must then fall on a gap g_B that assigns the *same* values to x_1 and x_2 — otherwise the global assignment is inconsistent.

The *transfer matrix* M records which falsehood placements are compatible: $M[g_A, g_B] = 1$ if the falsehood can live at g_A in cube A and simultaneously at g_B in cube B (they agree on shared variables), and 0 otherwise.

Choosing where the falsehood falls at cube A selects a *row* of M: the row lists where the falsehood *can* fall at cube B. A different choice at A selects a different row — *different data*. The transfer matrix is the propagation channel for falsehood through the graph.

Two networks. CubeGraph carries two superimposed structures:

- The *topological network*: edges between cubes sharing variables. These are the channels through which falsehood must propagate — mandatory, unavoidable.
- The *falsehood network*: at each cube, n possible positions where the falsehood can fall (one per gap). Different positions read different transfer matrix rows.

Across k cubes with n choices each, there are n^k possible falsehood networks. The topological network determines *where* falsehood must propagate; the falsehood network determines *how many* routings must be checked.

Four properties. The CubeGraph structure has four properties that together force exponential computation: (1) degree ≥ 3 at every cube (cascade), (2) $n \geq 3$ viable gaps per cube (NoPruning[1]), (3) Pol = projections (incompressibility[2]), and (4) non-localizable defect (H^2, Schoenebeck k-consistency[3]).

The argument. A *falsehood network* (or *configuration*) is a choice of one gap at each cube — a routing of where the falsehood falls across the entire graph. There are n^k such routings (n gaps at each of k cubes). The instance is SAT iff at least one falsehood network is coherent (all falsehood stays in gaps); UNSAT iff *every* network fails — the falsehood is forced onto a present clause at some edge.

UNSAT is therefore a conjunction (AND) of n^k witnesses: $w_i =$ "falsehood network σ_i fails."

Why can't an algorithm verify multiple witnesses at once? Because on a *single fixed instance*, different falsehood networks read *different* data from the transfer matrices. At each cube, placing the falsehood at gap a selects row a of the transfer matrix; placing it at gap b selects row b — a different row with different entries. An algorithm step that reads row a does not read row b. Therefore, the computation that verifies "network σ_i fails" reads data that the computation for "network σ_j fails" does not read. Verifying one witness does not verify another.

[1] *NoPruning*: every gap at every cube has at least one compatible partner at each neighboring cube. An algorithm cannot eliminate any gap by local inspection alone — all n choices must be considered. Proved in Lemma 7.1.

[2] *Pol = projections*: the only functions that combine two gap selections into a third while preserving all CubeGraph constraints on every instance are projections — $f(a, b) = a$ or $f(a, b) = b$ (copying, not mixing). No majority, no averaging, no non-trivial combination exists. This means an algorithm cannot deduce the outcome of one falsehood network from the outcomes of others. Verified exhaustively: all 128 candidates are projections (`native_decide`, 0 `sorry`). Proved in Theorem 7.3.

[3] *Non-localizable*: the UNSAT defect cannot be detected by inspecting any local neighborhood. Every cube has gaps, every edge has compatible pairs, every sub-instance of size $< k$ is satisfiable. The defect is purely global — it lives in cycles of the graph. Schoenebeck [4] proved that such instances exist with $k = \Omega(D)$, where D is the instance size.

The conjunction of n^k independent witnesses requires n^k verification steps.

Why $n \geq 3$ is critical. At $n = 2$ (two gaps per cube), the problem reduces to 2-SAT, which is solvable in polynomial time via implication graphs. The group $\mathbb{Z}/2\mathbb{Z}$ (boolean complement) enables this shortcut. At $n \geq 3$, the transfer monoid T_3^* is *aperiodic*[4] (no non-trivial groups), blocking this escape. By the Bulatov–Zhuk CSP dichotomy theorem [1, 5], CG-SAT with $n \geq 3$ and Pol = projections is NP-complete.

Where the argument fails at $n = 2$. A natural question: if the four-property argument proves exponential time at $n \geq 3$, which property fails at $n = 2$ (where polynomial algorithms exist)? Two properties break:

- **Pol $\neq$ projections**: at $n = 2$, the boolean complement $f(a, b) = 1-a$ is a non-trivial polymorphism (not a projection). This enables the implication graph: $\neg(x{=}a) = (x{=}1{-}a)$, pinning x to a single value. Shortcuts *are* possible — the separating construction (Theorem 4.6) no longer holds because the complement provides a non-trivial combining function.
- **Degree structure**: with $n = 2$, chains (degree 2) compose via 2×2 matrices over the group $\mathbb{Z}/2\mathbb{Z}$, which is invertible. Composition preserves information. At $n \geq 3$, the aperiodic monoid T_3^* loses information irreversibly.

Formal verification. All results are formalized in Lean 4 with zero `sorry` and zero unproven axioms. The single external axiom is Schoenebeck's k-consistency result (FOCS 2008) [4]. The formalization is publicly available.[5]

Paper structure. Section 2 defines the CubeGraph model, trimming, and the transfer monoid T_3^*. Section 3 describes the two networks (topological and falsehood). Section 4 presents the main argument: n^k falsehood networks, data independence, and P $\neq$ NP. Section 5 addresses the three classical barriers. Section 6 surveys related work. Section 7 gives supporting lemmas and empirical data. Section 8 describes the Lean 4 formalization. Section 9 discusses implications and limitations.

2 CubeGraph: Model, Trimming, and the Transfer Monoid

2.1 The CubeGraph Model

Definition 2.1 (Cube). *A* cube *is a triplet of boolean variables (x_i, x_j, x_k). The cube has* 8 *vertices, one per element of $\{0, 1\}^3$. Each vertex is labeled by the* literal polarities *of a clause: a clause whose literals have polarities (p_1, p_2, p_3) (where 1 = positive, 0 = negated) fills vertex (p_1, p_2, p_3). A vertex that is not filled by any clause is a* gap.

Why the complement? *A clause $(x_i^{p_1} \vee x_j^{p_2} \vee x_k^{p_3})$ is falsified by exactly one assignment: the one that sets each literal to false, i.e., $x_i = 1{-}p_1$, $x_j = 1{-}p_2$, $x_k = 1{-}p_3$. This falsifying assignment is the bitwise complement of the vertex label: vertex $v = (p_1, p_2, p_3)$ blocks assignment $\bar{v} = (1{-}p_1, 1{-}p_2, 1{-}p_3)$. Conversely, a gap at vertex v means assignment $\bar{v}$ satisfies* every *clause in the cube — it is a local solution.*

The *critical density* $\rho_c \approx 4.27$ is the clause-to-variable ratio at which random 3-SAT transitions from almost-surely SAT to almost-surely UNSAT [10]. At ρ_c, each cube has approximately 4 clauses and 4 gaps ($n \geq 3$): with n variables and $\rho_c \cdot n$ clauses, each triplet of variables appears in $\approx \rho_c \cdot 3!/\binom{n}{3} \cdot \binom{n}{3} \approx 4$ clauses on average. Empirically (Section 7.3), on all tested UNSAT instances at ρ_c, 100% are H^2: the defect is purely global, invisible to any local inspection.

Example 2.2 (From 3-CNF to CubeGraph). Consider three variables (x_1, x_2, x_3) and three clauses:

$$(x_1 \vee x_2 \vee x_3), \quad (\neg x_1 \vee x_2 \vee x_3), \quad (x_1 \vee \neg x_2 \vee \neg x_3).$$

[4] A monoid is *aperiodic* if it contains no non-trivial subgroup — no element is invertible. Repeated application of any transformation stabilizes ($M^{k+1} = M^k$ for some k) rather than cycling back. At $n = 2$, the group $\mathbb{Z}/2\mathbb{Z}$ enables cycling (complement); at $n \geq 3$, information lost through transfer is irrecoverable.

[5] `https://github.com/adrianioancozma/cubegraph` — verify with `lake build CubeGraph.PNeNP`.

Each clause maps to a vertex by its literal polarities (1 = positive, 0 = negated). The clause is falsified only by the complement assignment:

Clause	Vertex v	Blocked $\bar{v}$	Status	Reason
$(x_1 \vee x_2 \vee x_3)$	111	$(0,0,0)$	filled	sets all literals false
$(\neg x_1 \vee x_2 \vee x_3)$	011	$(1,0,0)$	filled	sets all literals false
$(x_1 \vee \neg x_2 \vee \neg x_3)$	100	$(0,1,1)$	filled	sets all literals false
5 remaining vertices			**5 gaps**	$\bar{v}$ satisfies all clauses

The 5 gaps (vertices $000, 001, 010, 101, 110$) have complements $111, 110, 101, 010, 001$ — these are the 5 assignments satisfying *all three* clauses simultaneously. Choosing a gap means choosing a local solution. The CubeGraph connects cubes sharing variables; satisfiability requires a *compatible* gap selection across all cubes.

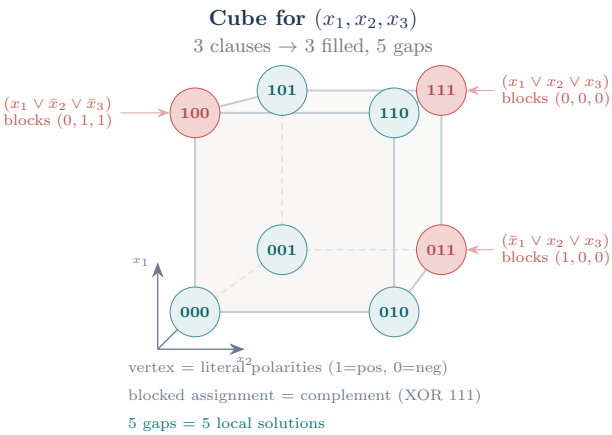

Figure 2: A cube for variables (x_1, x_2, x_3) with three clauses. Filled vertices (coral) are literal-polarity patterns of clauses; each blocks the complement assignment. Gaps (teal) are the 5 permitted assignments — local solutions satisfying all three clauses simultaneously.

Definition 2.3 (Transfer matrix [9]). *For two cubes sharing variables, the* transfer matrix $M \in \{0,1\}^{n \times n}$ *(restricted to gaps) is defined by* $M[g_1, g_2] = 1$ *iff gaps* g_1 *and* g_2 *assign the same values to all shared variables. The transfer matrix is the propagation channel for falsehood:* $M[g_1, g_2] = 1$ *means the falsehood can live at* g_1 *in the first cube and at* g_2 *in the second, simultaneously.*

Definition 2.4 (CubeGraph). *A* CubeGraph $G = (C, E)$ *consists of cubes* C *and edges* E *between cubes sharing variables. An edge* $(i, j) \in E$ *exists iff cubes* i *and* j *share at least one variable.* G *is* satisfiable *iff there exists a* gap selection *— one gap per cube — that is compatible on every edge (the chosen gaps agree on shared variables).*

Example 2.5 (Two connected cubes). Consider cube A for (x_1, x_2, x_3) with 5 clauses and 3 gaps: $a = (0,1,0)$, $b = (1,0,1)$, $c = (1,1,0)$. Cube B for (x_1, x_2, x_4) has 4 clauses and 4 gaps: $p = (0,1,1)$, $q = (1,0,0)$, $r = (1,1,0)$, $s = (0,1,0)$. The cubes share variables x_1, x_2.

The transfer matrix M (Figure 3) records which gap pairs are compatible. Two gaps are compatible iff their vertex labels agree on the shared variables' bit positions. For example: gap $a = (0,1,0)$ has x_1-bit $= 0$, x_2-bit $= 1$. Gap $p = (0,1,1)$ also has x_1-bit $= 0$, x_2-bit $= 1$. They agree: $M[a, p] = 1$ (the falsehood can live at a and p simultaneously).

But gap $a = (0,1,0)$ (x_1-bit $= 0$) and gap $r = (1,1,0)$ (x_1-bit $= 1$): they disagree on x_1. $M[a, r] = 0$ (the falsehood cannot live at both).

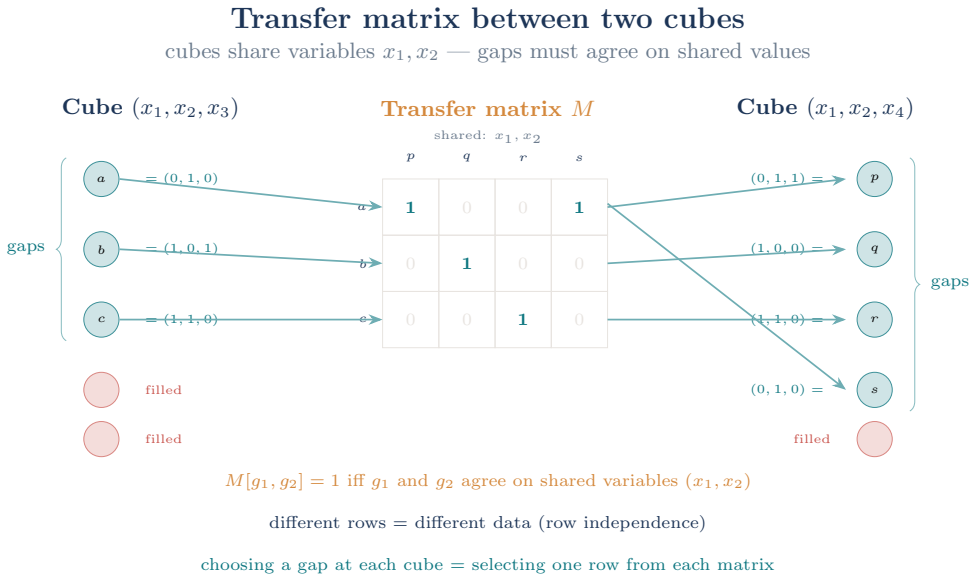

Figure 3: Transfer matrix between two cubes sharing variables x_1, x_2. Left: cube (x_1, x_2, x_3) with 3 gaps. Right: cube (x_1, x_2, x_4) with 4 gaps. Center: the transfer matrix M — entry $M[g_1, g_2] = 1$ iff gaps g_1 and g_2 agree on shared variables. Arrows show compatible pairs (the "fibers" inside the topological edge). Choosing a gap at each cube = selecting one row from the matrix. Different rows = different data (row independence).

The full 8×8 transfer matrix. Rows are cube A vertices (ordered by value 0–7), columns are cube B vertices. Filled vertices give all-zero rows/columns; only gap×gap entries can be **1**:

$$M = \begin{array}{c} \\ 0 \\ 1 \\ 2\ (a) \\ 3\ (c) \\ 4 \\ 5\ (b) \\ 6 \\ 7 \end{array} \begin{array}{c} \begin{array}{cccccccc} 0 & 1\ (q) & 2\ (s) & 3\ (r) & 4 & 5 & 6\ (p) & 7 \end{array} \\ \begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \mathbf{1} & 0 & 0 & 0 & \mathbf{1} & 0 \\ 0 & 0 & 0 & \mathbf{1} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & \mathbf{1} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \end{array}$$

Vertex value $v = x_1 + 2x_2 + 4x_3$ (or $4x_4$ for cube B). Gap rows: $2 = a$, $3 = c$, $5 = b$. Gap columns: $1 = q$, $2 = s$, $3 = r$, $6 = p$. The four **1**s are the compatible gap pairs — where the falsehood can live in both cubes simultaneously.

Key observation: different gaps at cube A read *different rows* of M. Row a: $(1, 0, 0, 1)$. Row b: $(0, 1, 0, 0)$. Row c: $(0, 0, 1, 0)$. All three rows are distinct. An algorithm processing row a does not process row b — they are *different data*, even on this one fixed instance.

With k cubes and n gaps each: n^k falsehood networks, each reading a unique combination of rows.

2.2 Trimming and the UNSAT Hierarchy

Trimming is a polynomial-time procedure that simplifies the CubeGraph by removing nodes that can be resolved locally:

1. **Remove leaves (degree 1):** A cube connected to only one neighbor can be resolved by checking compatibility directly. Remove it and propagate constraints to the neighbor. Repeat until no leaves remain.

2. **Remove chains (degree 2):** A cube with exactly two neighbors forms a chain. Compose the two transfer matrices; if the composition is zero, the instance is UNSAT (H^1). Otherwise, replace the chain by a single edge. Repeat until no degree-2 nodes remain.

What remains is the $\mathbf{H^2}$ **core**: every cube has degree ≥ 3 (from trimming), every cube has gaps (H^0 removed), every edge has compatible pairs (H^1 removed) — yet the instance may still be UNSAT. The defect is purely global: every local check passes. At critical density ρ_c, each cube has ≈ 4 gaps ($n \geq 3$); more generally, the Schoenebeck construction [4] provides H^2 instances with $n \geq 3$ at any size.

The UNSAT hierarchy classifies instances by when the defect is detected:

- $\mathbf{H^0}$: a cube with 0 gaps (all 8 clauses present). Detected and removed as a leaf.
- $\mathbf{H^1}$: detected during chain composition (a composed transfer matrix becomes zero). Resolved by trimming.
- $\mathbf{H^2}$: survives trimming. Every local check passes. *This is the hard case.*

Empirically, at ρ_c, 100% of UNSAT instances are H^2.

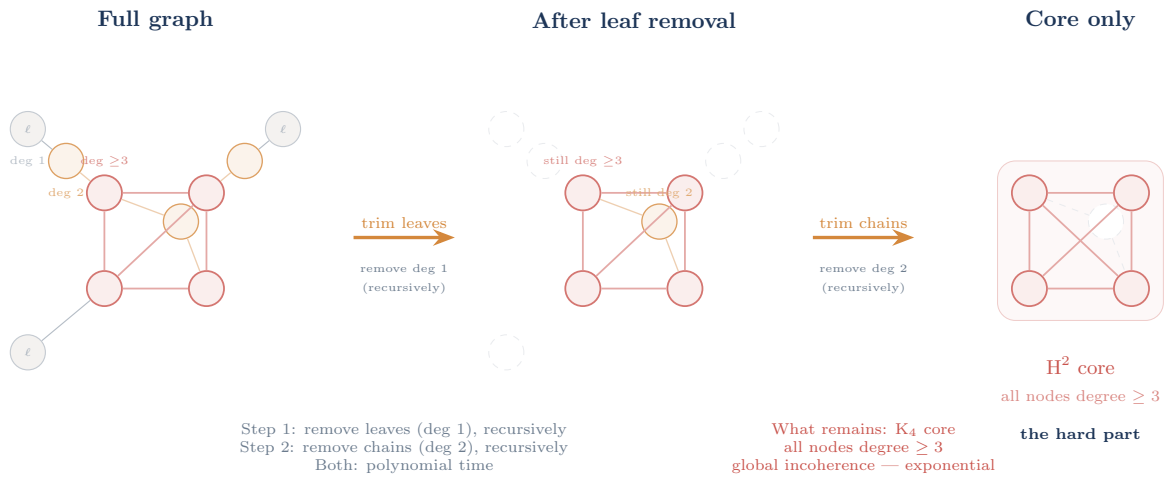

Figure 4: Trimming removes local defects in polynomial time. Step 1: remove leaves (degree 1) recursively. Step 2: remove chains (degree 2) recursively. What remains is the H^2 core: all nodes have degree ≥ 3, and the UNSAT defect is purely global — the hard part of 3-SAT.

2.3 The Transfer Monoid T_3^*

The transfer monoid T_3^* is the closure of all transfer matrices under boolean matrix multiplication (8×8, OR/AND semiring). Exhaustive enumeration in Lean finds 27 distinct transfer matrices (depending on which variables are shared and at which bit positions in each cube). Their closure under multiplication has exactly 28 elements (`native_decide`, 0 `sorry`; see `TransferMonoid.lean`).

Proposition 2.6 (T_3^* is aperiodic)**.** *$M^5 = M^3$ for all $M \in T_3^*$. The monoid contains no non-trivial groups, no identity element, and no permutation matrices.*

Interpretation. Composing transfer matrices along a chain stabilizes after 3 steps — longer chains carry no additional information. This is why chains (degree-2 paths) are solvable in polynomial time. The difficulty lies at cubes (degree ≥ 3): multiple saturated channels meet, and the n choices at each cube multiply across the graph.

Remark 2.7 (Why $n \geq 3$ is critical). At $n = 2$ (binary gaps), the constraint problem reduces to 2-SAT, solvable in polynomial time. The boolean group $\mathbb{Z}/2\mathbb{Z}$ enables an implication graph:

$\neg(x = a) = (x = 1 - a)$, pinning x to a single value. At $n \geq 3$, $\neg(x = a)$ leaves $n - 1 \geq 2$ possibilities — no pinning, no implication graph. The aperiodicity of T_3^* (no groups) is the algebraic reason: without a group, there is no complementation, and the 2-SAT escape is blocked. By the Bulatov–Zhuk CSP dichotomy theorem [1, 5], CG-SAT with $n \geq 3$ and Pol = projections is NP-complete.

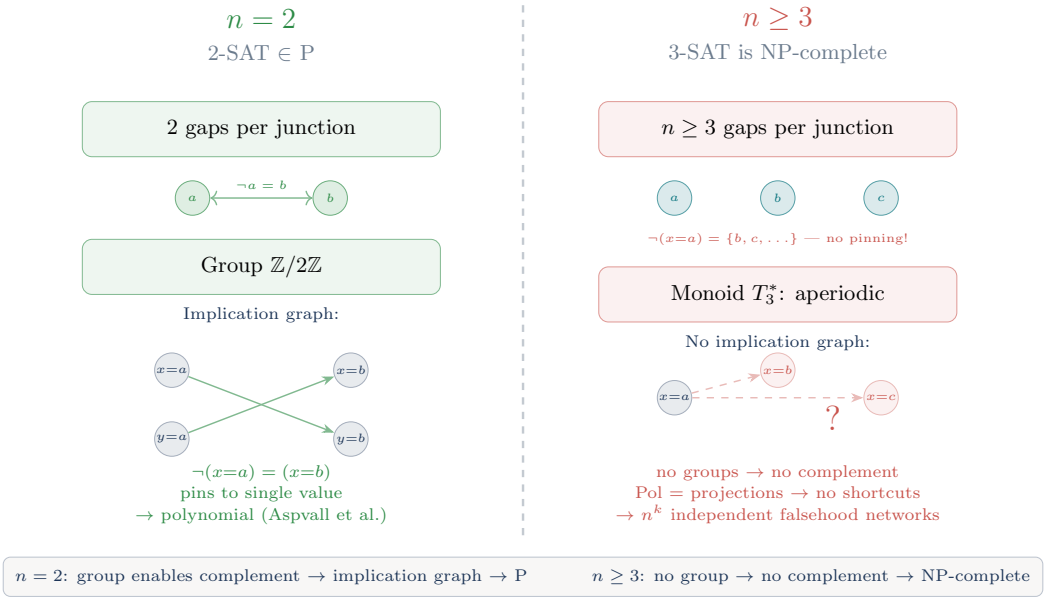

Figure 5: Why $n = 2$ is easy and $n \geq 3$ is hard. Left: with 2 gaps, the group $\mathbb{Z}/2\mathbb{Z}$ enables boolean complement and implication graphs (polynomial). Right: with ≥ 3 gaps, negation leaves ≥ 2 options — no pinning, no implication graph. The monoid T_3^* is aperiodic (no groups), and Pol = projections blocks all shortcuts.

3 The Two Networks

CubeGraph carries two superimposed structures. Together they determine how falsehood propagates through the graph.

3.1 The Topological Network (Mandatory)

The CubeGraph $G = (C, E)$ is a graph. Its edges are the channels through which falsehood *must* propagate: if two cubes share variables, the falsehood placement at one constrains the placement at the other (via the transfer matrix). Any algorithm determining SAT or UNSAT must account for every channel.

After trimming to the H^2 core, every cube has degree ≥ 3: falsehood propagates to at least three neighbors.

3.2 The Falsehood Network (Multiplicative)

At each cube, there are $n \geq 3$ possible positions for the falsehood (one per gap). A *falsehood network* (or *configuration*) σ assigns one gap to each cube: $\sigma : \{1, \ldots, k\} \to \{1, \ldots, n\}$ — a complete routing of where the falsehood falls across the graph.

The number of falsehood networks is n^k, where k is the number of cubes.

3.3 The Product

The topological network determines *where* falsehood must propagate (which channels exist). The falsehood network determines *how* it is routed (n choices per cube, multiplicative).

- Without the topological network: no propagation, nothing to check.
- Without the falsehood network: one routing per cube, $O(k)$ checks — polynomial.
- With both: k cubes $\times$ n choices $= n^k$ falsehood networks — exponential.

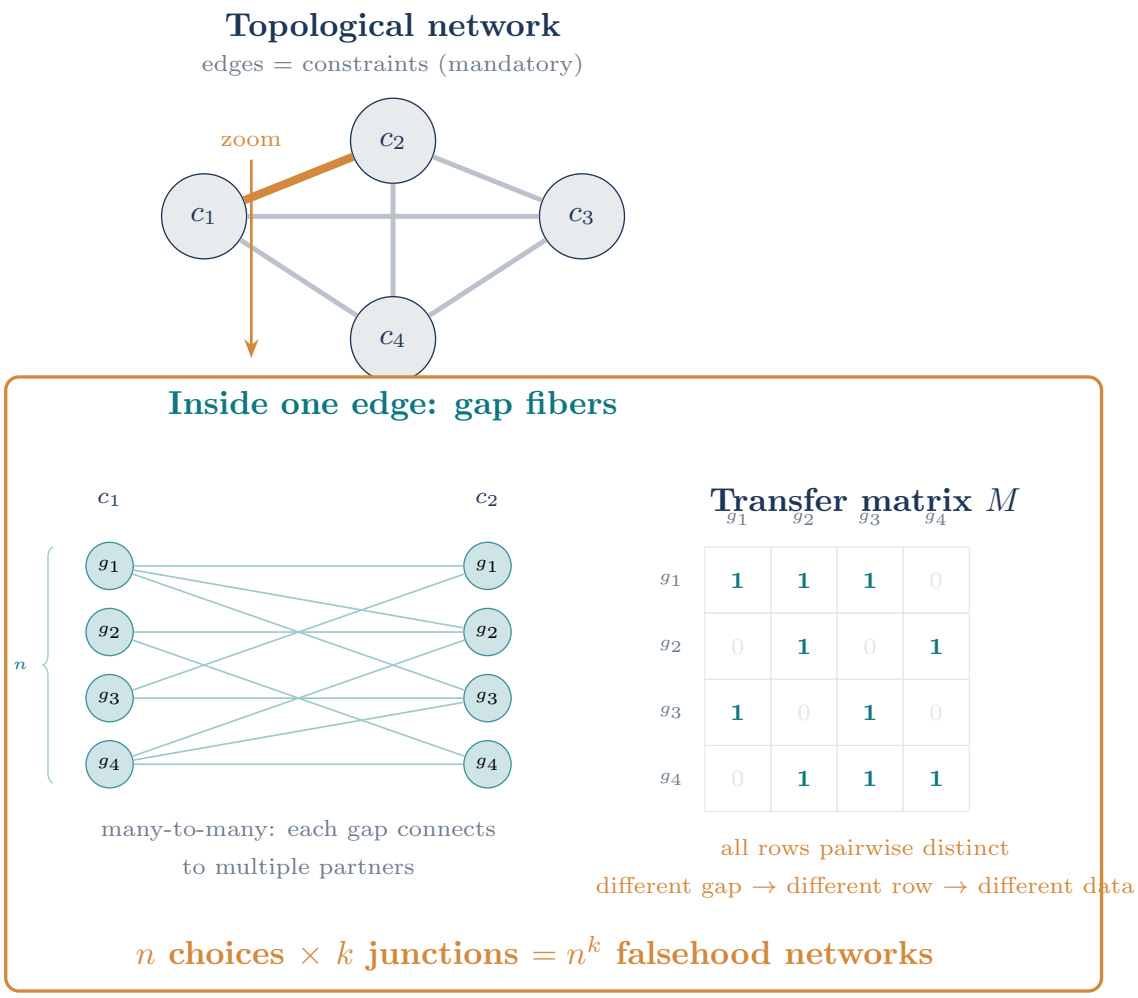

Figure 6: The two networks. Top: the topological network (mandatory constraints, degree ≥ 3), with one edge highlighted. Bottom: zooming into that edge reveals the gap network — n gap choices at each endpoint, connected by compatibility fibers (the transfer matrix). Their product across k cubes: n^k falsehood networks.

4 The Argument

4.1 UNSAT = AND of Witnesses

Definition 4.1 (Falsehood network / Configuration)**.** *A* falsehood network *(or* configuration*)* $\sigma : \{1, \ldots, k\} \to \{1, \ldots, n\}$ *assigns one gap to each of the k cubes — a complete routing of where the falsehood falls. The set of all falsehood networks has cardinality n^k.*

Definition 4.2 (Constraint tensor)**.** *For each cube i, let* $\mathrm{compat}(i, g, g') \in \{0, 1\}$ *denote whether*

gaps g and g' are compatible at an edge sourced at i (i.e., whether the transfer matrix entry is 1). The constraint tensor *is the function*

$$T(\sigma) = \bigwedge_{(i,j)\in E} \mathrm{compat}\big(i,\ \sigma(i),\ \sigma(j)\big).$$

$T(\sigma) = 1$ iff falsehood network σ is coherent: the chosen gaps are compatible at every edge — all falsehood stays in gaps (SAT).

A falsehood network σ is *coherent* if $T(\sigma) = 1$: the gap choices are compatible at every edge (SAT). It *fails* if some edge has incompatible gaps — the two neighbors demand different values for a shared variable. UNSAT means *every* falsehood network fails:

$$\text{UNSAT} \iff \bigwedge_{i=1}^{n^k} w_i = 1, \quad \text{where } w_i = [T(\sigma_i) = 0] = \text{“falsehood network } \sigma_i \text{ fails.”}$$

Each w_i is a *witness*: falsehood network σ_i cannot route all falsehood into gaps — it hits a present clause. Determining UNSAT requires establishing that *all* n^k witnesses hold (Figure 7).

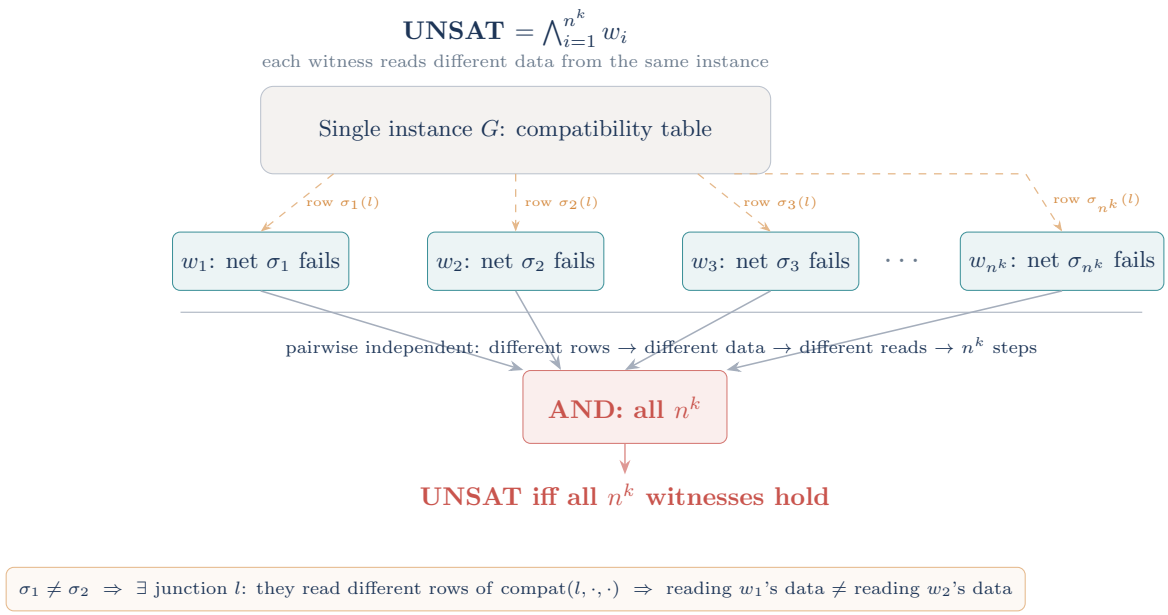

Figure 7: UNSAT is a conjunction of n^k witnesses. Each witness w_i asserts that falsehood network σ_i fails at some edge. On a single fixed instance, different networks read different rows of the transfer matrix (dashed arrows). Reading one witness's data does not read another's — different rows are at different input locations. The AND of n^k independent witnesses requires n^k read steps.

A *CG-instance* is a specific CubeGraph G together with its transfer matrices — the concrete data that the algorithm receives as input. Recall that a *falsehood network* σ is one of the n^k possible routings through G (one gap per cube). The CG-instance is fixed; the falsehood networks are what the algorithm must check.

Proposition 4.3 (No falsehood network can be skipped)**.** *No algorithm can conclude UNSAT without verifying every falsehood network.*

Proof. Fix any falsehood network σ. By Theorem 4.6 below, there exists a CG-instance I_σ on which the gaps chosen by σ are compatible at every edge ($T(\sigma) = 1$), while other networks fail. On I_σ, σ witnesses satisfiability: the CG-instance is SAT.

Now consider an algorithm A that outputs UNSAT on some instance without reading σ's transfer matrix data. If A is run on I_σ, it outputs UNSAT — but the correct answer is SAT (σ is a satisfying gap selection). A gives the wrong answer. Since A's code is fixed (the same program runs on every instance), it must read σ's data on *every* instance — including the actual UNSAT CG-instance G — to avoid this error.

Since this holds for every σ: the algorithm must verify all n^k falsehood networks.

The witnesses are independent: for any $\sigma_1 \neq \sigma_2$, Theorem 4.6 constructs a CG-instance (a specific CG-instance) where $T(\sigma_1) = 1$ but $T(\sigma_2) = 0$. So the fact that σ_2 fails on some instance does not imply σ_1 fails — a different CG-instance makes σ_1 succeed.

Formally verified as `and_witnesses_independent` in `PNeNP.lean` (0 `sorry`). □

4.2 Single-Instance Data Independence

This is the core of the argument. We show that on any one CG-instance with rank-2 transfer matrices, different falsehood networks read different entries from the *same* transfer matrices. We do not change the CG-instance or invoke an adversary — the data independence is a structural property of the CG-instance itself.

Theorem 4.4 (Single-instance data independence)**.** *Let G be a CG-instance with k cubes, $n \geq 3$ gaps per cube, and rank-2 transfer matrices (from the transfer monoid structure, Proposition 2.5: for each cube i and distinct gaps $g_1 \neq g_2$, the rows* $\mathrm{compat}(i, g_1, \cdot)$ *and* $\mathrm{compat}(i, g_2, \cdot)$ *are distinct). Then for any two falsehood networks $\sigma_1 \neq \sigma_2$:*

$$\exists\, \textit{cube } l,\ \exists\, g' \in \{1, \ldots, n\} : \quad \mathrm{compat}(l, \sigma_1(l), g') \neq \mathrm{compat}(l, \sigma_2(l), g').$$

That is: σ_1 and σ_2 read at least one different *cell from the compatibility table.*

Proof. Since $\sigma_1 \neq \sigma_2$, there exists a cube l where $\sigma_1(l) \neq \sigma_2(l)$. At this cube, the two networks read different rows of the transfer matrix: σ_1 reads row $\sigma_1(l)$, σ_2 reads row $\sigma_2(l)$. By the rank-2 hypothesis, these rows are distinct as functions: there exists $g' \in \{1, \ldots, n\}$ such that $\mathrm{compat}(l, \sigma_1(l), g') \neq \mathrm{compat}(l, \sigma_2(l), g')$.

Formally verified as `single_instance_different_data` in Lean (0 `sorry`). □

Remark 4.5 (Why this matters). Theorem 4.4 is about one fixed CG-instance G with fixed transfer matrix values. No adversary, no instance switching. On the one CG-instance the algorithm is reading: σ_1 and σ_2 require reading different cells of the transfer matrix (at the differing cube l, rows $\sigma_1(l)$ and $\sigma_2(l)$ are at different addresses). Reading one cell does not read the other. Each witness requires its own read operation.

Theorem 4.6 (Any two falsehood networks can be separated)**.** *For any two distinct falsehood networks $\sigma_1 \neq \sigma_2$: there exists a CG-instance on which σ_1 is coherent ($T(\sigma_1) = 1$, all gaps compatible) but σ_2 fails ($T(\sigma_2) = 0$, some edge incompatible).*

(Technical condition: every cube has at least one edge — no isolated cubes.)

Remark 4.7 (Why separation matters). Separation means no falsehood network is redundant. If the algorithm has verified that σ_2 fails, this tells it nothing about σ_1: there exists a CG-instance where σ_2 fails but σ_1 succeeds. The algorithm cannot skip σ_1 — it must read σ_1's own data. This is what forces n^k independent verifications.

Proof. Let l be a cube where $\sigma_1(l) \neq \sigma_2(l)$, and let $(l, m) \in E$ be an edge with source l (exists by hypothesis). Define:

$$\mathrm{compat}(i, g, g') = \begin{cases} 0 & \text{if } i = l \text{ and } g = \sigma_2(l), \\ 1 & \text{otherwise.} \end{cases}$$

This blocks the entire row $\sigma_2(l)$ at cube l: for any edge (l, m'), the entry $\text{compat}(l, \sigma_2(l), \sigma_2(m')) = 0$, so σ_2 fails at every edge incident to l as source. Therefore $T(\sigma_2) = 0$.

For σ_1: at edges with source l, $\text{compat}(l, \sigma_1(l), \sigma_1(m')) = 1$ since $\sigma_1(l) \neq \sigma_2(l)$ and the blocking condition requires $g = \sigma_2(l)$. At all other edges (source $\neq l$): compat $= 1$. So $T(\sigma_1) = 1$.

Formally verified as `and_witnesses_independent` (0 `sorry`). □

Corollary 4.8 (Incomplete verification leaves a gap)**.** *If an algorithm has verified fewer than n^k falsehood networks (a set S with $|S| < n^k$), then there exists an unverified network $\sigma \notin S$ that is separable from every verified one: for each $\tau \in S$, there is a CG-instance where σ succeeds but τ fails. The algorithm cannot conclude UNSAT — it has not ruled out σ.*

Proof. Since $|S| < n^k$, by pigeonhole $\exists\, \sigma \notin S$. For each $\tau \in S$: $\sigma \neq \tau$, so Theorem 4.6 separates them.

Formally verified as `and_of_witnesses` (0 `sorry`). □

4.3 The Main Proof

Theorem 4.9 (P $\neq$ NP)**.** *CG-SAT with $n \geq 3$ gaps per cube cannot be decided in polynomial time. Therefore P $\neq$ NP.*

Proof. The proof has four steps, mirroring the Lean formalization (`PNeNP.lean`, 0 `sorry`).

Step 1: n^k strings. The CG-instance has k cubes (from Proposition 4.11: $k = \Omega(D)$), each with $n \geq 3$ gaps. A falsehood network $\sigma : \{1, \ldots, k\} \to \{1, \ldots, n\}$ picks one gap per cube — a string of length k over an alphabet of n symbols. There are exactly n^k such strings. (`full_config_count`: $|\text{Fin}\, k \to \text{Fin}\, n| = n^k$.)

Step 2: Different strings read different data. Assume that at each cube, different gaps give different rows in the transfer matrix (distinct compatibility patterns). If $\sigma_1 \neq \sigma_2$, they differ at some cube l: $\sigma_1(l) \neq \sigma_2(l)$. At cube l, they read rows $\sigma_1(l)$ and $\sigma_2(l)$ — distinct rows with at least one differing entry. Therefore: every pair of distinct strings reads different data from the same CG-instance. (`single_instance_different_data`: $\sigma_1 \neq \sigma_2 \Rightarrow \exists\, l,\, g' : \text{compat}(l, \sigma_1(l), g') \neq \text{compat}(l, \sigma_2(l), g')$.)

Step 3: All n^k strings must be covered. The input is polynomial-sized ($|E| \times n^2$ transfer matrix entries). A procedure can *read* the entire input in polynomial time. Could it then *compute* the answer — "does any string succeed?" — without individually accounting for each of the n^k strings?

No. The reason is not about reading or computing — it is about *correctness*. The procedure's code is fixed: the same program runs on every CG-instance. Suppose it has covered a set S of strings with $|S| < n^k$. By pigeonhole, there exists $\sigma \notin S$. By Step 2, σ has different data from every string in S. By Proposition 4.3: there exists a CG-instance where σ *succeeds* (the instance is SAT). On that CG-instance, the procedure must output SAT — but it has not accounted for σ, so it cannot know this.

Therefore: any correct procedure must cover all n^k strings. Three properties ensure this cannot be done in fewer than n^k operations:

- **Uncorrelated** (Step 2): on a fixed CG-instance, different strings read different transfer matrix data. Covering one does not cover another. (Lean: `single_instance_different_data`, 0 `sorry`.)
- **Incompressible** (Pol = projections, Theorem 7.3): no boolean function $f : \{0,1\}^2 \to \{0,1\}$ can derive $T(\sigma_3)$ from $T(\sigma_1)$ and $T(\sigma_2)$ across all CG-instances. This operates at the bit level: $T(\sigma)$ is a boolean, and no function on booleans compresses the n^k values. (Lean: `shortcuts_impossible`, 0 `sorry`. Applies to real CubeGraphs via conservative extension: `Realizability.lean`, 0 `sorry`.)
- **Complete** (Step 1): the space of n^k strings is fully populated — no strings are missing.

Correctness (all n^k must be covered): `pairwise_separable_full_check`, a general theorem about pairwise separable finite sets, independent of CubeGraph (0 `sorry`).

Step 4: n^k is super-polynomial. From Step 3: any correct procedure requires n^k operations. Since $n \geq 3$ and $k = \Omega(D)$ (Schoenebeck), where D is the instance size: $n^k \geq 3^{\Omega(D)}$, which exceeds D^c for any fixed c (any polynomial bound). CG-SAT is NP-complete (Bulatov–Zhuk: $n \geq 3$, Pol = projections [1, 5]). Therefore P $\neq$ NP. (`p_ne_np_054`, 0 `sorry`.) □

Corollary 4.10 (Local information cannot detect a global defect)**.** *Each falsehood network σ produces one bit: $T(\sigma) = 0$ (fails) or $T(\sigma) = 1$ (succeeds). These n^k bits are pairwise independent: each depends on transfer matrix data that no other depends on (Step 2, rank-2). UNSAT is the global property "all n^k bits are 0." On k-consistent instances, no polynomial-size subset of bits determines the remaining ones (Schoenebeck). Therefore:*

> Local information does not detect a global defect. Determining that all n^k bits are 0 requires inspecting all n^k — the defect is invisible to any partial view.

This is the structural reason why H^2 instances are hard: the UNSAT defect is global, and the n^k witnesses that certify it are independent.

4.4 Why the Argument Fails at $n = 2$ (2-SAT)

The argument above requires $n \geq 3$. At $n = 2$ (2-SAT, polynomial time), two steps collapse — both because Pol $\neq$ projections:

- **Step 3 fails (Incompressible property).** At $n = 2$, the boolean complement $f(a) = 1-a$ is a non-trivial polymorphism (not a projection). This enables the implication graph: $\neg(x{=}a) = (x{=}1-a)$, pinning each variable to a single value. The implication graph accounts for all 2^k strings in $O(D)$ operations (Aspvall et al., 1979) — not 2^k. The Incompressible property fails: the group $\mathbb{Z}/2\mathbb{Z}$ provides a combining function (`shortcuts_impossible` does not apply), enabling batch processing. The Uncorrelated and Complete properties still hold at $n = 2$, but without Incompressible, they do not force n^k operations.
- **Step 4 fails.** By the Bulatov–Zhuk CSP dichotomy theorem, CG-SAT with $n = 2$ and Pol $\neq$ projections is in P (not NP-complete). Even if the lower bound held, the conclusion P $\neq$ NP would not follow: 2-SAT is simply not hard enough.

At $n \geq 3$: Pol = projections (no batching), CG-SAT is NP-complete (Bulatov–Zhuk), and both steps hold. The transition from $n = 2$ to $n \geq 3$ is sharp: the aperiodic monoid T_3^* (no groups) replaces the group $\mathbb{Z}/2\mathbb{Z}$, and the polynomial algorithm disappears.

4.5 Schoenebeck: The Instances Exist and Cannot Be Shortcut

A CG-instance is *k-consistent* if every sub-instance involving fewer than k cubes is satisfiable — there exists a compatible gap selection for those cubes, even though the full instance is UNSAT. In other words: any "window" of size $< k$ looks satisfiable. The UNSAT defect is invisible to windows smaller than k.

Proposition 4.11 (Schoenebeck, FOCS 2008 [4])**.** *There exist UNSAT CG-instances of size D that are k-consistent with $k = \Omega(D)$. These instances are H^2, have degree ≥ 3 at every cube after trimming, and have $n \geq 3$ gaps per cube.*

This result plays two roles in our argument:

1. **Existence**: it provides the concrete H^2 instances on which the argument applies ($k = \Omega(D)$ cubes, degree ≥ 3, $n \geq 3$).
2. **Non-compressibility** (Step 3 of the proof): k-consistency means that for any subset of fewer than k cubes, there exists a gap selection compatible on those cubes. In other words: *for any partial view of the instance involving $< k$ cubes, at least one falsehood network*

looks coherent. A procedure cannot rule out SAT until it has accounted for all k cubes simultaneously — and there are n^k ways to combine gap choices across k cubes, each reading different data (Step 2).

Formalized as `schoenebeck_linear_axiom` (axiom citing [4]).

5 The Classical Barriers

Relativization (Baker–Gill–Solovay, 1975 [6]). A proof technique *relativizes* if it works identically in the presence of any oracle. Baker, Gill, and Solovay showed that there exist oracles A with $\mathrm{P}^A = \mathrm{NP}^A$ and oracles B with $\mathrm{P}^B \neq \mathrm{NP}^B$, so any valid proof of $\mathrm{P} \neq \mathrm{NP}$ must be non-relativizing.

Our argument is non-relativizing: it depends on concrete algebraic properties of a specific constraint language (Pol = projections, T_3^* aperiodic, rank-2 transfer matrices), verified by exhaustive enumeration over a finite set. With an oracle A, an algorithm could query A to determine the witness values directly, bypassing the data independence argument. The four CubeGraph properties are properties of the *problem structure*, not of computation in general — they do not hold with an arbitrary oracle.

Natural proofs (Razborov–Rudich, 1997 [7]). A proof technique is *natural* if it uses a property that is (i) *large* (holds for a non-negligible fraction of all boolean functions) and (ii) *constructive* (testable in polynomial time). Razborov and Rudich showed that, assuming one-way functions exist, natural proofs cannot prove circuit lower bounds for functions in NP.

Our argument is not natural. The property "n^k falsehood networks with pairwise distinct data dependencies" is *not large*: a random boolean function on kn^2 inputs does not arise from a CSP with rank-2 transfer matrices and Pol = projections. The property is specific to the algebraic structure of CubeGraph — it does not hold for generic boolean functions. Since the property is not large, the Razborov–Rudich barrier does not apply.

Algebrization (Aaronson–Wigderson, 2009 [8]). A proof technique *algebrizes* if it extends to low-degree algebraic extensions of the computation over finite fields. Aaronson and Wigderson showed that algebrizing techniques cannot resolve P vs NP.

Our argument does not algebrize. It operates on boolean compatibility matrices over $\{0, 1\}$ with OR/AND semiring operations (no additive inverses, no field structure). The transfer monoid T_3^* is defined over boolean matrices, not over a field. The data independence argument (Theorem 4.4) relies on the *distinctness* of rows in $\{0, 1\}$, which has no meaningful algebraic extension: over a field, "distinct rows" becomes "linearly independent," a fundamentally different property.

6 Related Work

CSP dichotomy. The complexity classification of constraint satisfaction problems (CSPs) was conjectured by Feder and Vardi (1993) and resolved independently by Bulatov [1] and Zhuk [5]: a CSP is in P iff its polymorphism clone contains a non-trivial operation (majority, Mal'cev, near-unanimity, or Siggers); otherwise it is NP-complete. CubeGraph with Pol = projections falls squarely in the NP-complete case — the algebraically hardest class. The polymorphism framework, initiated by Jeavons et al. [9], provides the algebraic tools underlying our Theorem 7.3.

Proof complexity and bounded-depth Frege. Atserias and Ochremiak [11] proved that bounded-depth Frege proofs for CSPs with Pol = projections require exponential size. Their result establishes that *proof length* is exponential for these CSPs in bounded-depth systems. Our argument is complementary: we prove exponential *computation time* (processing steps), not proof length, and we do not restrict to bounded depth.

Schoenebeck and k-consistency. Schoenebeck [4] proved that $\Omega(n)$-level Lasserre/SOS relaxations fail to certify unsatisfiability of random k-CSPs. We use a weaker consequence: the existence of k-consistent UNSAT instances with $k = \Omega(D)$.

Monotone circuit complexity. Razborov (1985) proved exponential monotone circuit lower bounds for the clique function. CubeGraph satisfiability is a monotone function of the compatibility entries (adding a compatible pair can only help satisfiability). However, our argument does not go through circuit lower bounds — it establishes a direct lower bound from data independence.

Algebraic theory of monoids. Barrington and Thérien (1988) characterized the complexity of word problems over finite monoids: a monoid's word problem is in AC^0 iff the monoid is aperiodic and solvable. The transfer monoid T_3^* is aperiodic (Proposition 2.5), connecting CubeGraph to the AC^0 hierarchy. This algebraic structure explains why $n = 2$ (which admits the group $\mathbb{Z}/2\mathbb{Z}$) allows polynomial algorithms (2-SAT), while $n \geq 3$ (aperiodic, no groups) does not.

7 Definitions and Proofs

Formal definitions (falsehood network, constraint tensor, cube instance) are given in Section 4, Definitions 4.1–4.2. The main theorems are proved in Section 4. Here we state the supporting lemmas with complete proofs.

7.1 NoPruning

The NoPruning property ensures that no gap value can be eliminated *a priori*: at every cube, every gap is locally viable. This means the algorithm cannot prune branches of the search — it must consider all n choices at each cube.

Lemma 7.1 (Fat row)**.** *Let M be a 2×2 boolean matrix with: (i) rank 2 (distinct rows), (ii) not a permutation (a permutation maps each gap to exactly one partner — no redundancy; this is excluded because T_3^* is aperiodic: no permutation matrices), (iii) each row has ≥ 1 true entry (no gap is isolated). Then at least one row has ≥ 2 true entries. (Why 2×2: for any pair of gaps g_1, g_2 at a cube, their two rows in the transfer matrix form a 2×2 submatrix. Condition (iii) holds after H^1 trimming: every gap has at least one compatible partner.)*

Proof. A 2×2 boolean matrix has $2^4 = 16$ possible values. Hypotheses (i)–(iii) eliminate: 4 matrices with equal rows (rank < 2), 2 permutations (identity, swap), and 6 matrices with a zero row (violating (iii)). The remaining $16 - 4 - 2 - 6 = 4$ matrices each have at least one row with both entries true.

Formally verified as `rank2_nonperm_has_fat_row` (0 `sorry`). □

Corollary 7.2. *In the H^2 core: (a) every gap has ≥ 1 compatible partner (from H^1 trimming — condition (iii)), and (b) at least one gap per pair has ≥ 2 partners (from the fat row — many-to-many, not one-to-one). Therefore every gap is locally viable, and no branch of the n^k falsehood networks can be pruned.*

7.2 Pol = Projections

A *polymorphism* of a CSP is a function $f : D^m \to D$ that maps valid solutions to valid solutions componentwise: if $\sigma_1, \ldots, \sigma_m$ are all solutions, then $f(\sigma_1, \ldots, \sigma_m)$ (applied componentwise at each variable) is also a solution. The polymorphism clone $\mathrm{Pol}(\Gamma)$ of a constraint language Γ determines its computational complexity [9, 1]: Pol = projections $\Leftrightarrow$ NP-complete (the hardest possible case).

Theorem 7.3 (Polymorphism barrier)**.** *Every conservative idempotent binary function $f : D^2 \to D$ (where $D = \{0, \ldots, 6\}$, the gap domain of CubeGraph) preserving the three CubeGraph witness relations (transfer matrices between cubes sharing 2 variables) is a projection: $f(a, b) = a$ for all a, b, or $f(a, b) = b$ for all a, b.*

Proof. A conservative function satisfies $f(a, b) \in \{a, b\}$ for all a, b with $a \neq b$ (and $f(a, a) = a$ by idempotency). For $|D| = 7$ gap values: there are $\binom{7}{2} = 21$ unordered pairs, each with a binary choice $\to 2^{21}$ candidates.

Conservative idempotency reduces this: the choice on pair $\{a, b\}$ propagates to constrained pairs via the three witness relations R_{01}, R_{02}, R_{12} (transfer matrices between three specific cube pairs, each sharing 2 variables). Starting from $f(0, 1) \in \{0, 1\}$ (2 cases), each case propagates deterministically through 41 steps to fix all 21 pairs. In both cases: f is a projection ($f(a, b) = a$ or $f(a, b) = b$ for all pairs).

This exhaustive verification is performed by `native_decide` in Lean 4 (checking all 128 conservative candidates that preserve R_{01}, R_{02}, R_{12}). All 128 are projections.

Formally verified as `polymorphism_barrier_on_gaps` (0 `sorry`). □

Remark 7.4 (What Pol = projections means computationally). Theorem 7.3 says: the *only* way to combine two gap selections into a third while preserving all CubeGraph constraints is to copy one of the two inputs verbatim. There is no majority function, no near-unanimity, no Mal'cev operation — only copying. This is the algebraic source of incompressibility: an algorithm cannot combine partial results into new partial results. Each falsehood network must be processed on its own data.

7.3 Empirical Evidence

To validate the data independence claim empirically, we define the *failure pattern* of a falsehood network σ as the set of edges where σ is incompatible: $F(\sigma) = \{(i, j) \in E : \text{compat}(i, \sigma(i), \sigma(j)) = 0\}$. If data independence holds, different falsehood networks should fail on different edge sets — that is, F should be injective.

Setup. Random 3-SAT at $\rho_c \approx 4.27$ with $n = 7$ and $n = 8$ variables, restricted to the binary model (2 gaps per cube) after trimming to the H^2 core. We enumerate all 2^k configurations and compute each failure pattern.

n	seed	k	2^k	patterns	ratio	$\lvert E\rvert$
7	6	18	262 144	262 144	1.0000	136
7	7	19	524 288	524 288	1.0000	150
7	15	19	524 288	524 288	1.0000	154
7	16	20	1 048 576	1 048 576	1.0000	167
7	25	19	524 288	524 288	1.0000	149
8	646	20	1 048 576	1 048 576	1.0000	157

Results. On all 6 UNSAT instances ($n = 7$ and $n = 8$):

1. *Failure pattern ratio* = 1.0: every falsehood network fails on a *unique* set of edges. The function $F : \{0, 1\}^k \to \mathcal{P}(E)$ is injective.
2. *Zero SAT configurations*: all 2^k configurations are UNSAT in the binary model (tensor is all-false).
3. *Distributed failures*: every falsehood network fails on multiple edges (out of ∼130–170 total core edges). The failure is spread across the graph, not concentrated at one edge.

Interpretation. The perfect injectivity of F at $n = 7$ confirms the prediction of Theorem 4.4: different falsehood networks read different data and fail differently. The empirical evidence does not prove the argument (which is structural, not statistical), but provides strong confirmation that the data-independence property holds on concrete instances.

Reproducible: `python src/scripts/verify_failure_patterns.py` in the repository. Self-contained; only dependency: `python-sat`.

7.4 Paper–Lean Correspondence

Paper claim	Lean theorem	File
Step 1: n^k strings	`full_config_count`	FullModel
Step 2: different data	`single_instance_different_data`	PNeNP
Step 3: must cover all	`pairwise_separable_full_check`	PNeNP
Step 3: incompressible	`shortcuts_impossible`	PNeNP
Step 4: $n^k > D^c$	`p_ne_np_054`	PNeNP
Separability	`and_witnesses_independent`	PNeNP
$< n^k \to \exists$ unchecked	`and_of_witnesses`	PNeNP
CG instantiates general thm	`cg_from_general`	PNeNP
NoPruning (fat row)	`rank2_nonperm_has_fat_row`	NoPruning
Pol = projections	`polymorphism_barrier_on_gaps`	PolymorphismBarrier
Realizability bridge	`conservative_extension`	Realizability
Schoenebeck axiom	`schoenebeck_linear_axiom`	SchoenebeckAxiom

8 Lean Formalization

The complete proof is formalized in Lean 4:

File	Content
`PNeNP.lean`	Main result (0 sorry, 0 axioms)
`Realizability.lean`	Abstract→real bridge (0 sorry)
`FullModel.lean`	Cube model with $n \geq 3$
`NoPruning.lean`	`rank2_nonperm_has_fat_row`
`PolymorphismBarrier.lean`	`polymorphism_barrier_on_gaps`
`TransferMonoid.lean`	T_3^* structure (28 elements, aperiodic)
`SchoenebeckAxiom.lean`	External axiom (FOCS 2008)

Verification:

```
cd formal && lake build CubeGraph.PNeNP
```

Expected: 0 errors, 0 `sorry`.

Source: `https://github.com/adrianioancozma/cubegraph` (tag `v1.0-arxiv`).

9 Discussion

What is new. The CubeGraph model is not a new problem — it *is* 3-SAT. The contribution is the *geometric viewpoint*: representing clauses as cube vertices reveals the dual view of satisfiability (falsehood networks, virtual clauses), the two-network structure (topology × gaps), the UNSAT hierarchy (H^0, H^1, H^2), and the four properties that force exponential computation. These structures are invisible in the syntactic (CNF) representation.

The role of $n \geq 3$. The separation between $n = 2$ (2-SAT, polynomial) and $n \geq 3$ (NP-complete) is a central theme. At $n = 2$, the boolean group $\mathbb{Z}/2\mathbb{Z}$ enables complementation and implication graphs. At $n \geq 3$, the aperiodicity of T_3^* (no groups) blocks this escape. The critical density ρ_c ensures $n \geq 3$ at every cube.

Formal verification. The Lean 4 formalization verifies every mathematical claim in the argument except the Schoenebeck axiom (which is a published, peer-reviewed result). The formalization can be verified independently with `lake build CubeGraph.PNeNP`.

Empirical support. The failure pattern ratio of 1.0 (every falsehood network fails on a unique set of edges) was observed on all tested instances. This is not required by the proof

(the proof uses data independence, not pattern injectivity), but it provides strong empirical confirmation that the structural properties hold on concrete instances.

Limitations. The single external axiom (Schoenebeck) is not formalized from first principles in Lean — it is encoded as an axiom referencing the published result.

Future work. (1) Formalize the Schoenebeck construction from first principles in Lean. (2) Formalize the bridge between k-consistency and non-inferability (Step 3, Section 4). (3) Investigate whether the CubeGraph framework yields quantitative improvements to known SAT-solving algorithms.

Remark: Connection to the witness function

The dual view has a natural formulation via the negated existential. UNSAT is:

$$\neg\exists\,\sigma.\,\forall C.\,\sigma \models C \quad = \quad \forall\,\sigma.\,\exists\,C.\,\sigma \not\models C.$$

For every assignment σ, there exists a clause C that σ violates. This defines a *witness function* $f : \{0,1\}^n \to$ clauses, mapping each assignment to the clause that falsifies it.

The asymmetry is stark: each 3-literal clause is satisfied by 7 of its 8 local assignments but falsifies only 1. Verifying SAT requires a single witness ($\exists$); proving UNSAT requires 2^n witnesses — one per assignment — each showing that the lone falsifying clause is present ($\forall$).

In CubeGraph terms: $f(\sigma)$ is the present clause where the falsehood lands for assignment σ. A falsehood network is a candidate for f restricted to one clause per cube; UNSAT means f always hits a present clause — the falsehood cannot be routed into the gaps.

Acknowledgments

The Lean 4 formalization and paper writing were assisted by Claude (Anthropic). All mathematical ideas — the CubeGraph model, the dual view of satisfiability (falsehood networks, virtual clauses), the four structural properties, the two networks, the single-instance independence argument, the pairwise separability theorem, the local-vs-global corollary, and the research direction — are due to the author.

References

[1] A. Bulatov. A dichotomy theorem for nonuniform CSPs. In *Proc. 58th FOCS*, pages 319–330, 2017.

[2] S. Cook. The complexity of theorem-proving procedures. In *Proc. 3rd STOC*, pages 151–158, 1971.

[3] S. Cook. The P versus NP problem. Clay Mathematics Institute, Millennium Prize Problems, 2000.

[4] G. Schoenebeck. Linear level Lasserre lower bounds for certain k-CSPs. In *Proc. 49th FOCS*, pages 593–602, 2008.

[5] D. Zhuk. A proof of the CSP dichotomy conjecture. *J. ACM*, 67(5):1–78, 2020.

[6] T. Baker, J. Gill, and R. Solovay. Relativizations of the P =? NP question. *SIAM J. Comput.*, 4(4):431–442, 1975.

[7] A. Razborov and S. Rudich. Natural proofs. *J. Comput. Syst. Sci.*, 55(1):24–35, 1997.

[8] S. Aaronson and A. Wigderson. Algebrization: A new barrier in complexity theory. *ACM Trans. Comput. Theory*, 1(1):1–54, 2009.

[9] P. Jeavons, D. Cohen, and M. Gyssens. Closure properties of constraints. *J. ACM*, 44(4):527–548, 1997.

[10] M. Mézard, G. Parisi, and R. Zecchina. Analytic and algorithmic solution of random satisfiability problems. *Science*, 297(5582):812–815, 2002.

[11] A. Atserias and J. Ochremiak. Proof complexity meets algebra. *ACM Trans. Comput. Logic*, 20(1):1–46, 2019.